

基于 0624 的现状分析：为什么 CaptureAssetImage 直连可行，经过 Harness 连续调用会超时

文件名使用英文；正文使用中文。

写作方式参考本机 `code-to-article` skill：先说明链路和问题，再把每个判断落到具体代码位置，最后给出足够明确的改动建议。

结论先行

现在最值得执行的改动不是继续把 timeout 调大，也不是继续改 Python 侧 SSE parser。当前证据指向一个更窄的位置：

`CaptureAssetImage` 已经在 UE 内部执行，截图也已经完成；丢的是 UE MCP Server 最终那一帧 SSE result 从第二次 `OnComplete` 进入 UE HTTPServer 后，到 socket 被 Harness 读到之前 这段链路。

你看到的 UE 日志：

```
1 LogModelContextProtocol: Running tool:
  'ToolsetRegistry.EditorAppToolset.CaptureAssetImage'
2 LogModelContextProtocol: Running ToolsetRegistry tool:
  'ToolsetRegistry.EditorAppToolset.CaptureAssetImage'
3 LogModelContextProtocol: [DIAG] Async callback ENTERED:
  'ToolsetRegistry.EditorAppToolset.CaptureAssetImage'
4 LogModelContextProtocol: [DIAG] About to OnComplete SSE result for
  'ToolsetRegistry.EditorAppToolset.CaptureAssetImage'
5 LogCore: Display: Tracing Screenshot "ScreenShot00037" taken with size: 1318
  x 630
```

证明三件事：

1. UE MCP Server 收到了 `tools/call`。
2. ToolsetRegistry 确实执行了 `CaptureAssetImage`。
3. 截图 delegate 已经回调，MCP 层准备把最终结果交给 HTTPServer。

但它没有证明三件事：

1. HTTPServer 已经进入最终帧的 `Beginwrite()`。
2. `writeBytes()` 已经把 SSE body 写进 TCP socket。
3. Harness 的 `response.aiter_lines()` 已经读到任何一行 SSE body。

因此现在的改动方向应该是：减少 Harness 触发 UE HTTPServer SSE 状态机竞态的机会，并加最小 instrumentation 证明最终帧到底卡在哪一层。

推荐先执行两个低风险改动：

1. `take_screenshot` 使用一个持久的专用 UE MCP session，并用 `asyncio.Lock` 串行化截图调用，不要每张截图都 `connect()` + `DELETE`。
2. 修正 passthrough 测试参数名，把 `assetPath` 改成 UE schema 使用的 `AssetPath`，否则透传测试结论不可信。

如果这两个改动能让连续截图稳定，说明根因就是 session/connection teardown 与 `CaptureAssetImage` 异步 SSE result 写出之间的竞态。后续再考虑 UE 侧的真正修复：把最终 SSE 写出改成 HTTPServer/ticker 安全的队列式 streaming，而不是从异步工具回调里直接重入 `FHttpRequestCallback`。

读者校准

这份文档按“你看过项目，但不应该被迫记住每一段 UE HTTPServer 细节”的读者来写。

所以前半部分先用人话解释：

- 现在到底有几条 HTTP/MCP 链路。
- 为什么“UE 打了 About to OnComplete”不等于“Harness 收到了结果”。
- TCP 对一条连接里的响应顺序有什么硬规则。
- 为什么 `GetSelectedAssets` 这类工具和 `CaptureAssetImage` 不能类比。

后半部分再给代码定位、可证伪预测和建议改动。

一、当前链路：直接 9000/8000 的口头说法需要先还原成真实拓扑

当前仓库默认配置里：

- UE MCP Server 默认在 `127.0.0.1:8000/mcp`。
- Harness MCP Server 默认监听 `127.0.0.1:9000/mcp`。

代码位置：

- `harness/config.py`：`ue_port = 8000`，`listen_port = 9000`。
- `tests/tool_verify_ue_vision.py`：直连 UE 使用 `http://127.0.0.1:8000/mcp`。
- `tests/tool_verify_harness_vision.py`：经过 Harness 使用 `http://127.0.0.1:9000/mcp`。

关键片段：

```
1 # harness/config.py
2 ue_port: int = 8000
3 listen_port: int = 9000
4
5 @property
6 def ue_base_url(self) -> str:
7     return f"http://{self.ue_host}:{self.ue_port}/mcp"
```

所以如果运行环境里确实是“9000 直连 UE、8000 经过 Harness”，那说明启动参数和仓库默认相反。本文后续不依赖端口号，而是按下面这两个角色说：

角色	含义	默认端口
直连 UE	外部 MCP client 直接访问 UE MCP Server	<code>8000/mcp</code>
经过 Harness	外部 MCP client 访问 Harness，再由 Harness 转发到 UE	<code>9000/mcp</code>

真正要分析的是：为什么直连 UE 能拿到 `CaptureAssetImage` result，而经过 Harness 连续调用后，UE 明明截图完成，Harness 却超时。

二、完整执行链路：Harness 不是简单“转发一包 HTTP”

经过 Harness 的一次 `take_screenshot`，不是一跳，而是两条 MCP 连接叠在一起：

```
1  外部 MCP client
2      -> Harness MCP Server
3          -> Harness 内部 UE-side McpClientSession
4              -> UE MCP Server
5                  -> ToolsetRegistry adapter
6                      -> EditorAppToolset.CaptureAssetImage
7                          -> FScreenshotRequest / OnScreenshotCaptured
8                              <- async result
9                                  <- final SSE result
10                             <- Harness 读取 UE SSE
11                                 <- Harness 返回外层 MCP tool result
12                                     <- 外部 client 收到结果
```

这条链路中，**超时发生在 Harness 读取 UE SSE 的阶段**，不是 Harness 已经收到 UE 结果之后再转发失败。

证据在 `harness/client.py` 的 `_read_sse_stream()`：它会在第一行 SSE body 到达时记录 `[sse-stream]` 首行到达。

关键片段：

```
1  async for line in response.aiter_lines():
2      line_count += 1
3      if line_count == 1:
4          logger.info("[sse-stream] id=%d 首行到达, 耗时=%.1fs, 内容: %s",
5                      request_id, t_elapsed, line[:120])
```

0624 handoff 中失败样本的关键现象是：

调用	Harness 是否看到 SSE 首行	结果
<code>GetSelectedAssets</code>	看到，几乎 0 秒	成功
<code>GetOpenAssets</code>	看到，几乎 0 秒	成功
<code>CaptureAssetImage</code> 第一次	看到，约 52 秒	成功
<code>CaptureAssetImage</code> 第二次	没看到	<code>ReadTimeout</code>

这说明 Python 端已经收到了 HTTP response header，进入了 SSE 读取状态，但最终 `data: ...` body 没到。

三、你可能误解的 TCP/HTTP 规则：Header 到了，不代表 Body 到了

`tools/call` 在 UE MCP Server 里不是普通 JSON response，而是 SSE event-stream。它分两次写：

1. 第一次写：先把 HTTP 200 header 发出去，body 为空，告诉客户端“这是一个 SSE 流，后面还有数据”。
2. 第二次写：工具完成后，再把 `data: {"result": ...}\n\n` 这种 SSE body 发出去。

UE 代码位置：`ModelContextProtocolServer.cpp` 的 `ProcessToolCallJsonRpcCall()`。

第一次写：

```
1 TUniquePtr<FHttpResponse> Response =
2   FHttpResponse::Create(FString(TEXT("")),
3   UE::ModelContextProtocol::ContentTypeEventStream);
4 Response->Headers.Add(TEXT("Connection"), { TEXT("keep-alive") });
5 EnumAddFlags(Response->Flags,
6   EHttpResponseFlags::MultipleWriteStream |
7   EHttpResponseFlags::HasAdditionalWrites);
8 OnComplete(MoveTemp(Response));
```

这次写让 Harness 的 `httpx.stream()` 返回，因此 Python 侧可以看到 `status_code == 200` 和 `content-type: text/event-stream`。

但此时 **没有 result**。

第二次写：

```
1 TUniquePtr<FHttpResponse> ServerResponse =
2   FHttpResponse::Create(
3     UE::ModelContextProtocol::Private::FormatSSEMessage(ResponseStr),
4     UE::ModelContextProtocol::ContentTypeEventStream);
5 EnumAddFlags(ServerResponse->Flags,
6   EHttpResponseFlags::MultipleWriteStream |
7   EHttpResponseFlags::SkipHeaderWrite);
8 UE_LOG(LogModelContextProtocol, Log, TEXT("[DIAG] About to OnComplete SSE
9   result for '%ls'", *ToolName));
10 OnComplete(MoveTemp(ServerResponse));
```

这一帧才是 Harness 等待的 `data: {"result": ...}`。

所以 TCP/HTTP 层面的重要事实是：

一次 HTTP response 可以先发 header，再很久以后发 body；客户端拿到 header 只说明连接进入了响应流，不说明最终结果已经到了。

这正是当前症状：Harness 已经进入 `aiter_lines()`，但没有任何 SSE line。

四、另一个 TCP 规则：同一条 HTTP/1.1 keep-alive 连接上，状态机必须严格按顺序走

UE HTTPServer 的 `MultipleWriteStream` 依赖一个状态机：

1. 请求读完后，连接进入 `AwaitingProcessing`。
2. 第一次 `OnComplete` 触发 `BeginWrite()`，连接进入 `writing`。
3. 因为带 `HasAdditionalWrites`，写完 header 后连接回到 `AwaitingProcessing`，等待下一次 `OnComplete`。

4. 最终帧不带 `HasAdditionalWrites`，写完后连接回到 `AwaitingRead`，表示这条 keep-alive 连接可以读下一个请求。

代码位置：`HttpConnection.cpp`。

```
1 | check(EHttpConnectionState::AwaitingProcessing == SharedThisPtr->GetState());
2 | Response->HttpVersion = ResponseVersionCapture;
3 | SharedThisPtr->BeginWrite(MoveTemp(Response), LastRequestNumberCapture);
```

写完后的状态切换：

```
1 | if (WriteContext.HasAdditionalWrites())
2 | {
3 |     ChangeState(EHttpConnectionState::AwaitingProcessing);
4 | }
5 | else
6 | {
7 |     ChangeState(EHttpConnectionState::AwaitingRead);
8 | }
```

这套机制在理想测试中是能跑通的。UE 里有 `FHttpServerSSECallbackReinvocationTest`，测试 open stream、mid-stream、final frame 三段式写法。

关键片段：

```
1 | Response->Flags =
2 |     EHttpServerResponseFlags::MultipleWriteStream |
3 |     EHttpServerResponseFlags::HasAdditionalWrites;
4 | OnComplete(MoveTemp(Response));
```

最终帧：

```
1 | Final->Flags =
2 |     EHttpServerResponseFlags::MultipleWriteStream |
3 |     EHttpServerResponseFlags::SkipHeaderWrite;
4 | (*StashedCallback)(MoveTemp(Final));
```

这说明问题不是“UE HTTPServer 完全不支持多次 `OnComplete`”。问题更窄：**真实** `CaptureAssetImage` 的异步完成时机、线程/帧循环、session 删除、连接关闭，与这个状态机发生了竞态。

五、为什么 `GetSelectedAssets` 能成功不能证明 `CaptureAssetImage` 也应该成功

这几个工具看起来都走 `tools/call`，但执行形态完全不同。

1. 瞬时工具：结果很快在同一段调用链里返回

`GetSelectedAssets`、`GetOpenAssets` 这类工具通常是读编辑器状态，执行时间短，基本没有跨帧等待。它们也使用 SSE，但第一次 header 和最终 result 之间间隔很短。

结果是：

```
1 HTTP header 到达
2 SSE body 很快到达
3 Harness 首行日志出现
4 工具返回
```

他们对 UE HTTPServer 状态机的压力很小。

2. CaptureEditorImage：同步截图，不依赖 viewport screenshot delegate

CaptureEditorImage() 在 EditorAppToolset.cpp 中是同步返回 FToolsetImage 的函数。

```
1 FToolsetImage UEditorAppToolset::CaptureEditorImage()
2 {
3     // Capture every visible slate window ...
```

它也可能耗时，但它不像 viewport screenshot 那样依赖 FScreenshotRequest::OnScreenshotCaptured() 以后再完成。

3. CaptureAssetImage(AssetPath="")：跨帧异步截图

空 AssetPath 会进入 viewport screenshot 路径：

```
1 if (AssetPath.IsEmpty() ||
2     CurrentLevelPackage == FPackageName::ObjectPathToPackageName(AssetPath))
3 {
4     return FViewportScreenshotCapture::Start(bShowUI).Get();
5 }
```

FViewportScreenshotCapture 做的是：

1. 配置当前 Level Editor viewport。
2. 订阅 FScreenshotRequest::OnScreenshotCaptured()。
3. 调用 FScreenshotRequest::RequestScreenshot(bShowUI)。
4. 等截图 delegate 以后再 Result->SetValue(Image)。

关键片段：

```
1 Handle = UE::ToolsetRegistry::FDelegateHandlerRaii::Create(
2     OnScreenshotCaptured,
3     OnScreenshotCaptured.AddLambda(
4         [This = AsShared().ToSharedPtr()](
5             int32 width, int32 Height, const TArray<FColor>& Colors) mutable
6         {
7             This->OnCaptured(width, Height, Colors);
8             This.Reset();
9         }));
10 FScreenshotRequest::RequestScreenshot(bShowUI);
```

完成时：

```
1  if (Image.SetFromBitmap(Final, Dimensions))
2  {
3      Result->SetValue(Image);
4  }
```

这就是“一步同步差异”的核心：

工具类型	执行形态	SSE 两次写之间的间隔	竞态风险
<code>GetSelectedAssets</code>	读取编辑器状态，几乎同步	极短	低
<code>CaptureEditorImage</code>	同步合成窗口截图	较短/中等	中
<code>CaptureAssetImage(AssetPath=“”)</code>	viewport 截图 delegate 异步完成	可跨帧、几十秒	高

`CaptureAssetImage` 的最终 SSE result 不是在最初处理 HTTP request 的同一段调用栈中自然返回，而是在截图完成以后，通过 future/continuation/adapter 再回到 MCP `onComplete`。这正是状态机最容易被连接关闭、session 删除、线程时机影响的地方。

六、Harness 为什么比直连更容易触发问题

直连 UE 的测试大致是：

```
1  initialize
2  notifications/initialized
3  load_toolset
4  CaptureAssetImage
5  CaptureEditorImage
6  GetSelectedAssets
7  CaptureAssetImage(asset)
8  close
```

Harness 的路径更复杂：

```
1  外部 client -> Harness initialize
2  Harness 启动时 -> UE initialize
3  Harness 可能 preload/list/load 多个 toolset
4  外部 client 调 take_screenshot
5  Harness 内部为截图新建 shot_client
6  shot_client initialize
7  shot_client notifications/initialized
8  shot_client CaptureAssetImage
9  shot_client close -> DELETE UE session
10 外部 client 再次调 take_screenshot
11 重复上面流程
```

关键代码在 `harness/verification/capturer.py`：

```

1 shot_client = _McpClientSession(shot_config)
2 try:
3     await shot_client.connect()
4     result = await shot_client.call_tool(
5         "ToolsetRegistry.EditorAppToolset.CaptureAssetImage",
6         {"AssetPath": path, "bShowUI": b_show_ui},
7     )
8     return parse_screenshot(result, max_width, max_height)
9 finally:
10    await shot_client.close()

```

`close()` 又会发 UE `DELETE` :

```

1 resp = await self._http.delete(
2     self._config.ue_base_url,
3     headers=self._build_headers(),
4 )

```

UE 收到 `DELETE` 后会直接移除 session:

```

1 const int32 RemovedCount = Sessions.RemoveAll([&SessionId](const
2     TSharedPtr<FModelContextProtocolSession>& InSession)
3 {
4     return InSession.IsValid() && (InSession->ID == SessionId);
5 });

```

这就是直连和 Harness 的真正差异: **Harness 多了大量 session/connection 生命周期动作, 而且这些动作和长耗时异步截图的 final SSE result 写出挤在同一个时间窗口里。**

直连成功, 只说明简单链路没有触发竞态; 不能证明复杂链路也安全。

七、现在不该继续相信的解释

解释 A: timeout 太短

不成立。0624 已经把 `sse_read_timeout` 统一到 120 秒, 并且截图 session 放大到 600 秒。失败时不是“多等一会儿就能到”, 而是首行 SSE body 根本没到。

解释 B: Python 还在用 `response.content` 阻塞整个 SSE

不成立。当前 `call_tool()` 对 `text/event-stream` 使用 `httpx.stream()` 和 `response.aiter_lines()`。

关键片段:


```

1  async with self._http.stream(
2      "POST",
3      self._config.ue_base_url,
4      json=payload,
5      headers=self._build_headers(),
6  ) as stream_response:
7      ...
8      if "text/event-stream" in content_type:
9          result = await self._read_sse_stream(rid, response)

```

这已经是正确的 SSE 消费方式。

解释 C: URL /mcp/mcp 拼错

不成立。当前 `ue_base_url` 已经是完整 URL，代码不再追加额外路径：

```

1  return f"http://{self.ue_host}:{self.ue_port}/mcp"

```

解释 D: UE 没执行工具

不成立。你的日志已经证明工具执行了，而且 `Tracing Screenshot taken` 证明截图已经完成。

解释 E: Harness 已经收到 UE result，只是没转发给外部 client

目前证据不支持。因为 `_read_sse_stream()` 没有看到首行 SSE。若 Harness 已经收到 UE result，至少应该出现 `[sse-stream]` 首行到达 或 收到 `result`。

八、最可疑断点：About to OnComplete 到 WriteBytes 之间

UE MCP Server 的日志现在只打到这里：

```

1  UE_LOG(LogModelContextProtocol, Log,
2      TEXT("[DIAG] About to OnComplete SSE result for '%ls'"), *ToolName);
3  OnComplete(MoveTemp(ServerResponse));

```

但真正把 bytes 写入 socket 的代码在 HTTPServer：

```

1  bool FHttpConnectionResponseWriteContext::WriteBytes(
2      const uint8* Bytes,
3      int32 BytesLen,
4      int32 &OutBytesWritten)
5  {
6      OutBytesWritten = 0;
7      bool bwriteSuccess = Socket->Send(Bytes, BytesLen, OutBytesWritten);

```

当前缺失的证据是：

1. 第二次 `OnComplete` 有没有进入 `FHttpConnection::Beginwrite()`。
2. 如果进入了，`writeContext.ResetContext()` 里 response flags/body length 是什么。
3. `WriteBytes()` 是否尝试写 final SSE body。
4. socket 是否返回 `SE_EWOULDBLOCK`、`SE_TRY_AGAIN`、其他错误，或一直写 0。

5. 写完后连接状态是否从 `Writing` 回到 `AwaitingRead`。

只要这几处补齐，就能把问题从“感觉是 UE/Harness 中间丢了”压缩成一个具体分支。

九、可证伪假设与预测

假设 1：最终 SSE frame 没有进入 UE HTTPServer 写流程

如果这是根因，那么日志会显示：

```
1 [DIAG] About to OnComplete SSE result
```

但看不到同一个 connection/request 的：

```
1 [DEBUG-cai-sse] BeginWrite final frame
```

这说明 `FHttpRequestCallback` 被调用时连接状态不对、callback 失效，或者 HTTPServer 内部直接丢弃/断言前返回。

推荐改动：UE 侧不要从异步工具回调直接重入 `OnComplete`，改成 game thread / HTTP tick 安全队列。

假设 2：最终 SSE frame 进入了写流程，但 socket 没写出去

如果这是根因，那么日志会显示：

```
1 [DEBUG-cai-sse] BeginWrite flags=MWS|SkipHeaderWrite bodyBytes=N
2 [DEBUG-cai-sse] WriteBytes body attempt=N written=0 err=...
```

或者多次 `written=0` 直到超时/连接销毁。

推荐改动：避开 callback-reinvocation streaming；使用 `StreamingBodyQueue` 或文件落盘返回路径。

假设 3：Harness 每次截图后的 `DELETE` 与 delayed callback 存在竞态

如果这是根因，那么把 `shot_client.close()` 暂时跳过，或者改成长生命周期截图 session，会显著提高连续截图成功率。

推荐改动：为 screenshot 建一个持久 UE MCP session，串行复用；不要每张截图后立即 `DELETE`。

假设 4：Harness 主动 `response.close()` 破坏 UE HTTPServer keep-alive 状态

当前 `call_tool()` 收到 result 后会显式 `await response.close()`：

```
1 result = await self._read_sse_stream(rid, response)
2 await response.close()
3 return result
```

如果第一次成功后客户端主动关闭连接导致 UE HTTPServer 没有完全复位，那么第二次同类 `MultipleWriteStream` 更容易失败。

预测：成功路径不主动 `aclose()`，而是让 server final frame 后自然回到 `AwaitingRead` / `close`，连续调用稳定性可能改善。

注意：这个实验要谨慎做，因为不 `aclose()` 也可能让 httpx 连接池持有旧连接。更低风险的实验是截图专用 session + 禁止连接复用/串行化。

十、我建议立刻执行的改动

改动 1：修正 passthrough 测试的参数名

当前 `tests/tool_verify_harness_passthrough.py`：

```
1 r = await session.call_tool(target, {"assetPath": "", "bshowUI": False})
```

但 UE tool schema 和直连测试使用的是：

```
1 {"AssetPath": "", "bshowUI": False}
```

这会让“透传模式是否触发同一问题”的测试结论变脏。应先改成：

```
1 r = await session.call_tool(target, {"AssetPath": "", "bshowUI": False})
```

这不是根因修复，但它是后续诊断的地基。地基歪了，后面每次实验都会让人多怀疑一个变量。

改动 2：截图用持久专用 session，不要每次截图 `connect()` + `DELETE`

当前 `capturer.capture()` 每次都创建独立 `McpClientSession`，finally 里 `close`。

建议改成：

- Harness 启动时或首次截图时创建一个 screenshot session。
- 所有 `take_screenshot` 通过同一个 session 调用 UE。
- 用 `asyncio.Lock` 保证同一时间只有一张截图。
- 仅在 Harness 关闭、UE reconnect、session 出错时释放/重建。

目标不是“让代码更优雅”，而是减少对 UE HTTPServer/SSE 状态机的刺激：

```
1 现在：
2 截图1: initialize -> CaptureAssetImage -> DELETE
3 截图2: initialize -> CaptureAssetImage -> DELETE
4 截图3: initialize -> CaptureAssetImage -> DELETE
5
6 建议：
7 截图session: initialize
8 截图1: CaptureAssetImage
9 截图2: CaptureAssetImage
10 截图3: CaptureAssetImage
11 Harness shutdown/reconnect: DELETE
```

如果这个改动让连续截图稳定，基本可以证明问题与 session/connection teardown race 强相关。

改动 3：UE 侧加最小写出 instrumentation

不要“到处打日志”。只加能区分分支的日志：

位置	日志内容	用来证明什么
ModelContextProtocolServer.cpp final callback	session id、request id、tool name、response bytes、IsInGameThread()、OnComplete 返回后日志	MCP 层是否正常提交 final frame
HttpConnection.cpp OnProcessingComplete	connection id、request number、state、flags	final frame 是否进入 HTTPServer
HttpConnection.cpp BeginWrite / Completewrite	state before/after、HasAdditionalWrites	状态机是否正确前进
HttpConnectionResponseWriteContext.cpp::WriteBytes	attempt bytes、written bytes、socket error	final body 是否真正写 socket
harness/client.py::_read_sse_stream timeout path	session id、request id、elapsed、headers	Python 是不是一直 0 body

统一前缀建议：[DEBUG-cai-sse]。调完后能一次 grep 清理。

改动 4：中期改成文件式截图返回

如果目标是稳定产品能力，而不是验证 UE MCP SSE 机制，那么更稳的是：

1 | UE 截图 -> 写 PNG 到临时文件 -> MCP result 只返回 JSON 路径 -> Harness 读文件/resize/vision

这会绕过最脆弱的组合：

- 长耗时异步截图
- SSE callback reinvocation
- 1MB+ base64 result
- HTTP keep-alive 状态机
- 连续 session 创建/删除

MCP 仍然用于触发截图，但不承载大体积图片 body。这个改动最能降低未来维护成本。

十一、为什么我认为现在值得改，而不是继续观察

继续观察的收益很低，因为已有证据已经排掉了大部分 Python 侧解释：

- URL 拼接已修。
- timeout 已拉长。
- _cancel_request 已停用。
- response.content 已换成 stream() + aiter_lines()。
- UE 日志证明工具执行完成。
- Harness 日志证明 SSE 首行没到。

当前剩下的是一个典型“跨层状态机 + 异步回调 + 连接生命周期”的问题。它不会靠再多等 120 秒变稳定；它只会在连续调用、截图慢、UE 编辑器负载高、连接池状态变化时继续随机/半稳定复现。

执行持久截图 session 的好处是：

- 1. 改动范围小，主要在 Harness。
- 2. 不需要马上修改 UE Engine HTTPServer。
- 3. 能直接验证 `DELETE/session churn` 是否是放大器。
- 4. 即使 UE 侧最终仍需修，这个改动也能减少生产路径上的故障概率。

执行 instrumentation 的好处是：

- 1. 不再争论“是不是 Harness 没转发”。
- 2. 可以精确看到 final SSE frame 到没到 `WriteBytes()`。
- 3. 下一步修 UE 还是修 Harness，会有明确证据。

十二、代码位置速查

区域	文件	关键位置	作用
Harness 配置	<code>harness/config.py</code>	<code>ue_port / listen_port / ue_base_url</code>	定义 UE 与 Harness 默认端口
Harness UE client	<code>harness/client.py</code>	<code>McpClientSession.call_tool()</code>	发送 UE <code>tools/call</code> ，读取 SSE
Harness SSE parser	<code>harness/client.py</code>	<code>_read_sse_stream()</code>	逐行读取 SSE，首行日志是关键证据
Harness 截图入口	<code>harness/server.py</code>	<code>take_screenshot</code> 分支	外部 MCP tool 调截图
Harness 截图实现	<code>harness/verification/capturer.py</code>	<code>capture()</code>	当前每次截图新建 session 并 close
Passthrough 测试	<code>tests/tool_verify_harness_passthrough.py</code>	<code>session.call_tool(...)</code>	当前 <code>assetPath</code> 参数名错误
UE MCP tools/call	<code>ModelContextProtocolServer.cpp</code>	<code>ProcessToolCallJsonRpcCall()</code>	两阶段 SSE: header + final result
UE MCP DELETE	<code>ModelContextProtocolServer.cpp</code>	<code>ProcessDeleteRequest()</code>	删除 MCP session
Toolset adapter	<code>ModelContextProtocolToolsetRegistryAdapter.cpp</code>	<code>FToolsetRegistryToolAdapter::RunAsync()</code>	把 ToolsetRegistry future 转成 MCP result
Viewport 截图	<code>EditorAppToolset.cpp</code>	<code>FViewportScreenshotCapture</code>	<code>CaptureAssetImage(AssetPath="")</code> 的异步截图路径
同步截图	<code>EditorAppToolset.cpp</code>	<code>CaptureEditorImage()</code>	同步合成 editor window image
UE HTTP 状态机	<code>HttpConnection.cpp</code>	<code>OnProcessingComplete / Beginwrite / CompleteWrite</code>	<code>MultipleWriteStream</code> 状态切换
UE socket 写	<code>HttpConnectionResponseWriteContext.cpp</code>	<code>WriteBytes()</code>	最终判断 bytes 是否写进 socket
UE SSE 测试	<code>HttpServerTests.cpp</code>	<code>FHttpServerSSECallbackReinvocationTest</code>	证明理想情况下多次 <code>OnComplete</code> 可行

十三、最终建议

我建议按这个顺序执行：

- 1. 先修 `tool_verify_harness_passthrough.py` 的 `AssetPath` 参数名。
- 2. 加 `[DEBUG-cai-sse]` 最小日志，尤其是 UE HTTPServer 的 `Beginwrite/WriteBytes`。
- 3. 把 `take_screenshot` 改成持久截图 session + `asyncio.Lock` 串行调用。
- 4. 用同一套测试连续跑：
 - 直连 UE viewport screenshot 两次。
 - Harness passthrough `CaptureAssetImage` 两次。
 - Harness `take_screenshot(mode="viewport")` 两次。
- 5. 如果持久 session 稳定，保留 Harness workaround，并把 UE 侧文件式截图作为中期稳定方案。

6. 如果仍失败，优先修 UE 侧 final SSE 写出路径：不要让异步截图 completion 直接重入 `FHttpRequestCallback`，改为队列式 streaming 或文件结果。

这不是“为了试试而改”。它直接瞄准当前证据留下的唯一高价值断点：**UE 已完成截图，但 final SSE result 没有稳定跨过 HTTPServer/socket 边界到达 Harness。**